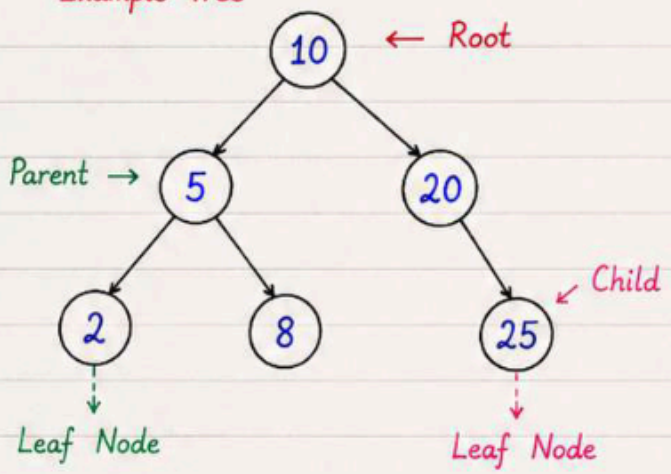
TREE & GRAPH HANDBOOK

1. INTRODUCTION TO TREE & TERMINOLOGY

★ What is a Tree ?

- A Tree is a non-linear data structure that consists of nodes.
- It has a hierarchical structure with a special node called Root.
- Every node (except root) has exactly one parent and can have zero or more children.

Example Tree



★ Terminology

- Root : The topmost node of the tree.
- Parent : A node that has one or more children.
- Child : A node that is a descendant of another node.
- Sibling : Nodes that share the same parent.
- Leaf Node : A node that has no children.
- Internal Node : A node that has at least one child.
- Level : The level of a node is the number of edges from root to that node (Root is at level 0).
- Depth of a node : Number of edges from root to that node.
- Height of a tree : Number of edges on the longest path from root to the deepest node.

Example (Above Tree)

- Root = 10
- Parent of 2 = 5
- Children of 5 = 2, 8
- Siblings of 2 = 8
- Leaf Nodes = 2, 8, 25
- Level of 25 = 2
- Height of Tree = 2

★ Tree Node Structure in Java

```
class Node {
    int data;
    Node left;      // reference to left child
    Node right;     // reference to right child
    Node(int data) {
        this.data = data;
        left = right = null;
    }
}
```

Left → smaller
Right → greater
(used in BST)

★ Applications of Trees

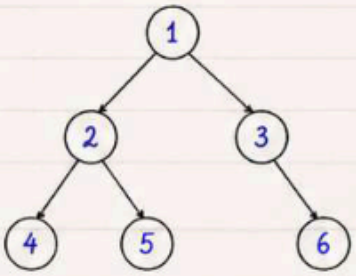
- ✓ File System (Folders & Subfolders)
- ✓ Organization Hierarchy
- ✓ Expression Trees (Compilers)
- ✓ Database Indexing (B-Trees)
- ✓ DOM (Document Object Model)
- ✓ Network Routing (Spanning Tree)

2. Binary Tree & Types of Binary Tree

★ Binary Tree

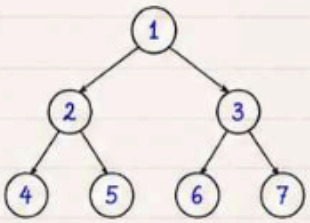
A tree in which each node can have at most two children: left child and right child.

General Binary Tree

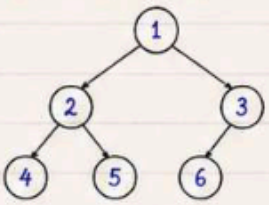


Types of Binary Tree (with examples)

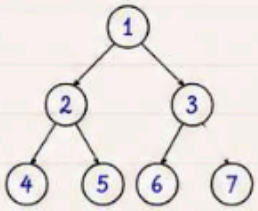
1. Full Binary Tree
(0 or 2 children)



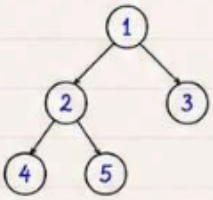
2. Complete Binary Tree
(All levels are completely filled except possibly last, filled left to right)



3. Perfect Binary Tree
(All levels full & all leaf nodes at same level)



4. Balanced Binary Tree
(Height difference between left and right subtree of every node is at most 1)

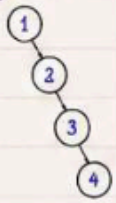


5. Skewed Binary Tree
(Every node has only one child)

Left Skewed



Right Skewed



★ Creating a Binary Tree Node (Java)

```
class Node {
    int data;
    Node left;      // reference to left child
    Node right;     // reference to right child

    Node(int data) {
        this.data = data;
        left = null;
        right = null;
    }
}
```

★ Ways to Create a Binary Tree

- Manually (hardcoding)
- Using Recursive Input
- Using Level Order Input

★ Basic Operations on Binary Tree

- Create
- Traverse
- Search
- Count Nodes
- Find Height
- Delete (in Binary Search Tree)

★ Time Complexity (for basic operations)

Operation	Best Case	Average Case	Worst Case
Search (in BST)	O(1)	O(log n)	O(n)
Insert (in BST)	O(1)	O(log n)	O(n)
Delete (in BST)	O(1)	O(log n)	O(n)

Applications

- Expression Trees
- Syntax Trees
- Decision Trees
- File Systems
- Game Trees
- AI (Minimax Tree)

3. Binary Search Tree (BST)

★ What is BST ?

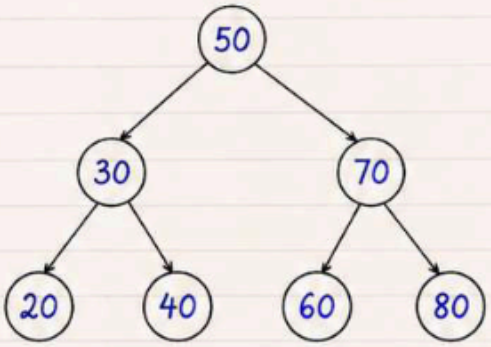
A Binary Tree in which for every node,

- All values in the **left** subtree are **smaller**.
- All values in the **right** subtree are **greater**.

★ Properties

1. Left < Node < Right
2. Inorder traversal of BST gives **sorted order**.
3. No duplicate values (we will not allow duplicates here).

Example BST



Inorder Traversal

20, 30, 40, 50, 60, 70, 80

(Sorted Order)

★ Insertion in BST (Algorithm)

1. If tree is empty → new node becomes root.
2. If value < root → insert in left subtree.
3. If value > root → insert in right subtree.
4. Repeat until a NULL position is found.

★ Java Program : Insert in BST

```
class Node {
    int data;
    Node left, right;
    Node(int data) {
        this.data = data;
        left = right = null;
    }
}
class BST {
    Node root;
    // Insert function
    Node insert(Node root, int key) {
        if (root == null) {
            return new Node(key);
        }
        if (key < root.data) {
            root.left = insert(root.left, key);
        } else if (key > root.data) {
            root.right = insert(root.right, key);
        }
        return root;
    }
}
```

★ Search in BST (Algorithm)

1. Start from root.
2. If key == root.data → return true.
3. If key < root.data → search in left subtree.
4. If key > root.data → search in right subtree.
5. If reach NULL → key not found.

★ Java Program : Search in BST

```
boolean search(Node root, int key) {
    if (root == null) {
        return false;
    }
    if (key == root.data) {
        return true;
    }
    if (key < root.data) {
        return search(root.left, key);
    } else {
        return search(root.right, key);
    }
}
```

★ Time Complexity

Operation	Average Case (Balanced BST)	Worst Case (Skewed BST)
Search	O(log n)	O(n)
Insert	O(log n)	O(n)
Delete	O(log n)	O(n)

★ Applications

- ✓ Dictionary / Set Implementation
- ✓ Database Indexing
- ✓ Range Queries
- ✓ Auto Suggestions
- ✓ Symbol Table
- ✓ File Systems

★ Note :

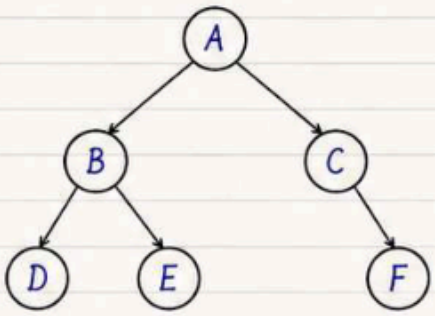
Inorder of BST → gives **sorted** sequence which is very useful.

4. Tree Traversals

★ Why Tree Traversal ?

- Traversal means visiting each node of the tree exactly once in a particular order.
- Used to process, search, copy, or delete nodes.
- Different traversals give different orders of nodes.

Example Binary Tree



Legend

Root = A
Children of A = B, C
Children of B = D, E
Child of C = F
Leaf Nodes = D, E, F
Height of Tree = 2

★ Types of Traversals

Traversal	Order (Rule)	Output (For Above Tree)	Type	Used When ?
1. Inorder (LNR)	Left → Node → Right	D, B, E, A, C, F	Depth First (DFS)	When tree is BST, it gives sorted order.
2. Preorder (NLR)	Node → Left → Right	A, B, D, E, C, F	Depth First (DFS)	Used to create a copy of tree.
3. Postorder (LRN)	Left → Right → Node	D, E, B, F, C, A	Depth First (DFS)	Used to delete the tree or free memory.
4. Level Order (BFS)	Level by Level from left to right	A, B, C, D, E, F	Breadth First (BFS)	Used in printing level wise or shortest path.

★ Recursive Implementation (Java)

Inorder (LNR)

```
void inorder(Node root) {
    if(root == null) return;
    inorder(root.left);
    System.out.print(root.data+" ");
    inorder(root.right);
}
```

Preorder (NLR)

```
void preorder(Node root) {
    if(root == null) return;
    System.out.print(root.data+" ");
    preorder(root.left);
    preorder(root.right);
}
```

Remember (Memory Trick)

Inorder → L N R
Preorder → N L R
Postorder → L R N
Level Order → Level by Level

★ Level Order Traversal (Using Queue)

```
void postorder(Node root) {
    if(root == null) return;
    Queue<Node> q = new LinkedList<>();
    q.add(root);
    while(!q.isEmpty()) {
        Node curr = q.poll();
        System.out.print(curr.data+" ");
        if(curr.left != null)  q.add(curr.left);
        if(curr.right != null) q.add(curr.right);
    }
}
```

Traversal Type	Time Complexity	Auxiliary Space
Inorder	O(n)	O(h)
Preorder	O(n)	O(h)
Postorder	O(n)	O(h)
Level Order	O(n)	O(w)

n → Number of nodes
h → Height of tree (For DFS)
w → Maximum width of tree (For BFS)

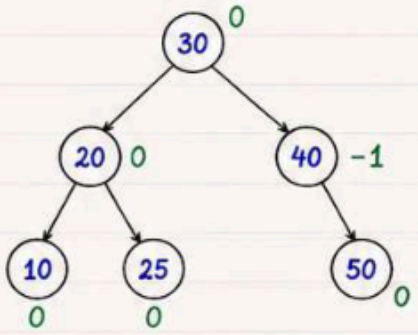
DFS uses recursion stack → O(h)
BFS uses queue → O(w)

5. AVL Tree (Self-Balancing BST)

★ Why AVL Tree ?

- In normal BST, tree can become skewed and operations take O(n).
- AVL Tree is a self-balancing BST where the height difference between left and right subtree of every node is at most 1.
- It maintains balance using rotations.

Example AVL Tree



Balance Factor (BF)

BF of a node
= Height(Left Subtree) − Height(Right Subtree)

For AVL Tree,
BF can be -1, 0 or +1

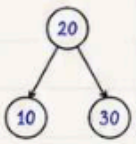
★ Rotations in AVL Tree

1. Right Rotation (LL Case)

Before



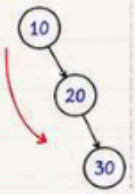
After



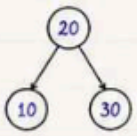
When BF of node = +2 and BF of left child >= 0

2. Left Rotation (RR Case)

Before



After



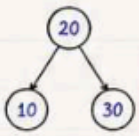
When BF of node = -2 and BF of right child <= 0

3. Left-Right Rotation (LR Case)

Before



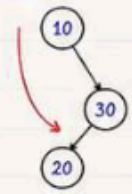
After



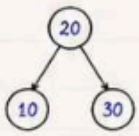
When BF of node = +2 and BF of left child = -1

4. Right-Left Rotation (RL Case)

Before



After



When BF of node = -2 and BF of right child = +1

★ AVL Tree Properties

1. It is a BST.
2. For every node, |BF| <= 1.
3. Height of AVL tree is O(log n).
4. Rotations ensure the tree remains balanced after every insertion or deletion.

★ Java Code : Get Height & Balance Factor

```
class Node {
    int data, height;
    Node left, right;
    Node(int data) {
        this.data = data;
        this.height = 1; // height of new node is 1
    }
}
```

```
// returns height of node
int height(Node node) {
    if (node == null) return 0;
    return node.height;
}
// returns balance factor of node
int getBalance(Node node) {
    if (node == null) return 0;
    return height(node.left) - height(node.right);
}
```

★ Insertion in AVL Tree (Steps)

1. Perform normal BST insertion.
2. Update height of ancestor nodes.
3. Calculate balance factor (BF) of ancestors.
4. If |BF| > 1, perform appropriate rotation (LL, RR, LR or RL).
5. Return the new root of the subtree.

★ Java Code : Right Rotation (LL Case)

```
Node rightRotate(Node y) {
    Node x = y.left;
    Node T2 = x.right;

    // Perform rotation
    x.right = y;
    y.left = T2;

    // Update heights
    y.height = Math.max(height(y.left), height(y.right)) + 1;
    x.height = Math.max(height(x.left), height(x.right)) + 1;

    // New root
    return x;
}
```

★ Time Complexity

Operation	Time Complexity
Search	O(log n)
Insert	O(log n)
Delete	O(log n)
Space Complexity	O(n)

★ Applications of AVL Tree

- Database indexing
- Memory management
- File systems
- Used in systems where balanced tree is must (e.g., real-time systems)

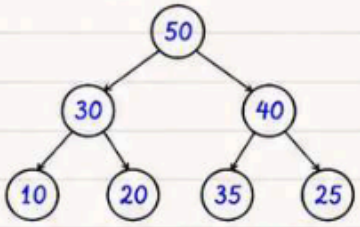
Note :
AVL trees are used where frequent search, insert and delete are required with guaranteed performance.

6. Heap (Introduction)

★ What is Heap ?

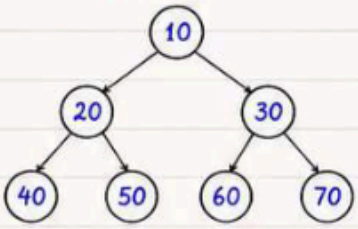
- A Heap is a complete binary tree that satisfies the Heap property.
- Heaps are used to implement Priority Queue efficiently.
- Two types of Heaps :
  - → Max Heap
  - → Min Heap

Max Heap (Parent ≥ Children)



Every parent node is greater than or equal to its children.

Min Heap (Parent ≤ Children)



Every parent node is smaller than or equal to its children.

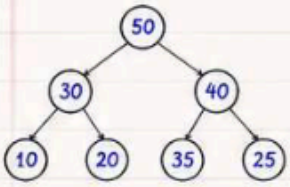
★ Heap as Array Representation

For 0-based indexing :

- Left child = 2*i + 1
- Right child = 2*i + 2
- Parent = (i - 1) / 2

Example :

Max Heap Tree



→

Array Representation

50	30	40	10	20	35	25
0	1	2	3	4	5	6

★ Heap Property

- Max Heap → Parent ≥ Children
- Min Heap → Parent ≤ Children

★ Applications of Heap

- ✓ Implementation of Priority Queue
- ✓ Heap Sort
- ✓ Dijkstra's Shortest Path Algorithm
- ✓ CPU Scheduling
- ✓ Top K frequent elements
- ✓ Median in a stream

Heap Insertion (Max Heap)

1. Insert the new element at the end of the heap.
2. Compare with parent. If greater, swap.
3. Repeat until heap property is satisfied.

Example : Insert 45

Initial Heap (Max Heap)

50	30	40	10	20	35	25
0	1	2	3	4	5	6

Step 1: Insert 45 at end

50	30	40	10	20	35	25	45
0	1	2	3	4	5	6	7

Step 2: 45 > 35 → swap

50	30	40	10	20	45	25	35

Step 3: 45 > 40 → swap

50	30	45	10	20	40	25	35

Step 4: 45 < 50 → stop

Final Heap

50	30	45	10	20	40	25	35

Heapify (Max Heap)

Used to convert a binary tree into a heap.

- Start from last non-leaf node.
- Compare with children.
- Swap with largest child.
- Repeat till root.

Example :

Given Array

10	20	5	30	40	15
0	1	2	3	4	5

Step 1: Start from index 2 (Node 5)

10	20	15	30	40	5

Step 2: Move to index 1 (Node 20)

10	40	15	30	20	5

Step 3: Move to index 0 (Node 10)

Final Max Heap

40	30	15	10	20	5

Java Code : Heap (Max Heap)

```
class MaxHeap {
    int arr[];
    int size;
    int capacity;

    MaxHeap(int cap) {
        capacity = cap;
        size = 0;
        arr = new int[capacity];
    }
    // Insert element
    void insert(int val) {
        if (size == capacity) return;
        arr[size] = val;
        int i = size;
        size++;
        // Percolate up
        while (i != 0 && arr[(i-1)/2] < arr[i]) {
            int temp = arr[i];
            arr[i] = arr[(i-1)/2];
            arr[(i-1)/2] = temp;
            i = (i-1)/2;
        }
    }
}
```

Time Complexity

Operation	Time Complexity (Max/Min Heap)
Insert	O(log n)
Delete (Extract Root)	O(log n)
Peek (Get Root)	O(1)
Heapify	O(n)

Note :

- Heap is always a complete binary tree.
- In Max Heap → largest element is at root.
- In Min Heap → smallest element is at root.
- Used in priority based operations.

7. Introduction to Graph

★ What is Graph ?

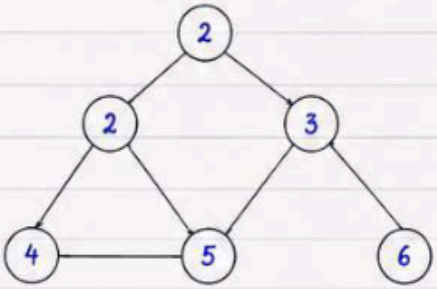
- A Graph is a non-linear data structure.
- It consists of a set of vertices (nodes) and edges
- that connect pairs of vertices.
- A graph G is represented as

$$G = (V, E)$$

where V = set of vertices

E = set of edges

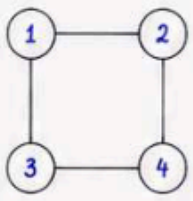
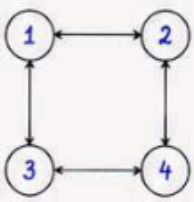
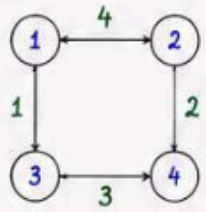
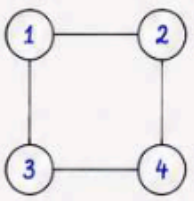
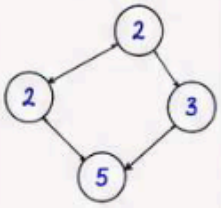
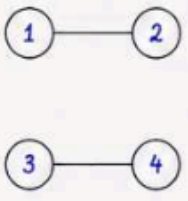
Example Graph (Undirected, Unweighted)



Terminology

- **Vertex (or Node) :** A point in the graph.
- **Edge :** A line connecting two vertices.
- **Degree of a Vertex :** Number of edges incident on the vertex.
- **Path :** A sequence of vertices connected by edges.
- **Cycle :** A path that starts and ends at the same vertex.

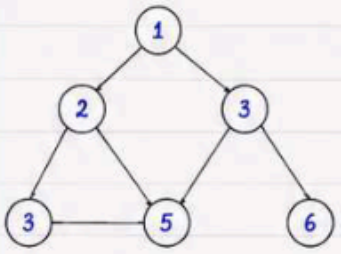
★ Types of Graph

1. Undirected Graph	2. Directed Graph (Digraph)	3. Weighted Graph	4. Unweighted Graph	5. Connected Graph	6. Disconnected Graph
					

★ Key Points

- In undirected graph, edges have no direction.
- In directed graph, edges have a direction from one vertex to another.
- In weighted graph, each edge has a weight or cost.
- A graph can be connected or disconnected.
- Graph may contain cycles or may be acyclic.

Degree Example (Using Above Graph)

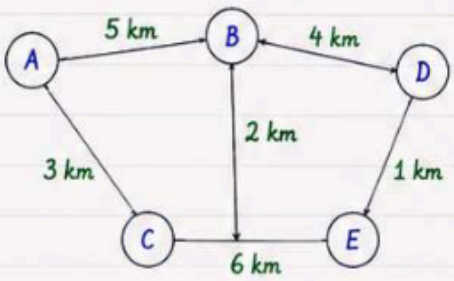


deg(1) = 2
deg(2) = 3
deg(3) = 3
deg(4) = 2
deg(5) = 4
deg(6) = 2

★ Applications of Graph

- Social Networks (Facebook, LinkedIn)
- Road Maps & GPS Navigation
- Web Link Structure (Google)
- Recommendation Systems
- Network Routing
- Scheduling & Task Dependencies
- Circuit Design

Real World Example



Notes :

- Graphs are very powerful to model relationships.
- They are widely used in AI, ML, and real world systems.
- Understanding graphs is important for many algorithms like BFS, DFS, Dijkstra, etc.

8. Graph Representation

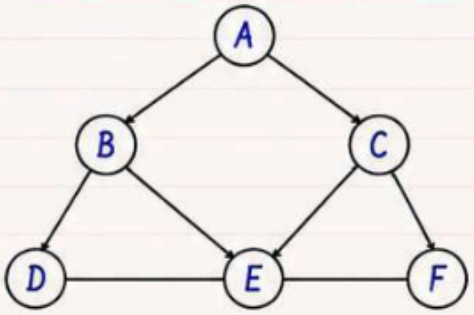
★ Ways to Represent a Graph

There are two most common ways to represent a graph :

1. Adjacency Matrix
2. Adjacency List

Choice depends on number of vertices (V) and edges (E).

Example Graph (Undirected, Unweighted)



Note :

- Number of Vertices, $|V| = 6$
- Number of Edges, $|E| = 8$
- Undirected Graph (Edge A-B is same as B-A)
- Unweighted Graph (All edges have equal weight)

1. Adjacency Matrix

- A 2D matrix of size V x V.
- matrix[i][j] = 1 (or weight) if edge exists between vertex i and j.
- matrix[i][j] = 0 if no edge exists.
- For undirected graph, matrix is symmetric.

Adjacency Matrix for Above Graph

	A	B	C	D	E	F
A	0	1	1	0	0	0
B	1	0	0	1	1	0
C	1	0	0	0	1	1
D	0	1	0	0	1	0
E	0	1	1	1	0	1
F	0	0	1	0	1	0

Space Complexity

$O(V^2)$

Best For

Dense Graph ($E \approx V^2$)

Java Code : Adjacency Matrix

```
int V = 6; // number of vertices
int[][] matrix = new int[V][V];

// add edge function for undirected graph
void addEdge(int u, int v) {
    matrix[u][v] = 1;
    matrix[v][u] = 1; // because undirected
}
```

2. Adjacency List

- An array (or HashMap) of size V.
- Each index i stores a list of all vertices adjacent to i.
- More space efficient for sparse graphs ($E << V^2$).
- Easier to iterate over neighbors.

Adjacency List for Above Graph

A : B → C
B : A → D → E
C : A → E → F
D : B → E
E : B → C → D → F
F : C → E

Space Complexity

$O(V + E)$

Best For

Sparse Graph ($E << V^2$)

Java Code : Adjacency List

```
int V = 6; // number of vertices
List<Integer>[] adj = new ArrayList[V];
// initialize list for each vertex
for (int i = 0; i < V; i++)
    adj[i] = new ArrayList<>();
// add edge function for undirected graph
void addEdge(int u, int v) {
    adj[u].add(v);
    adj[v].add(u); // because undirected
}
```

★ Comparison :

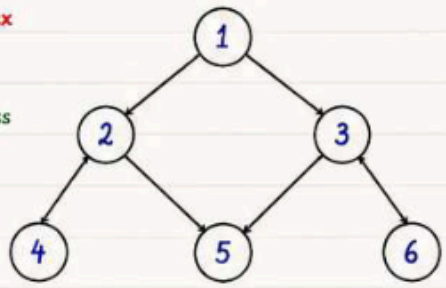
Feature	Adjacency Matrix	Adjacency List	Which to Use ?
Space Complexity	$O(V^2)$	$O(V + E)$	List for sparse graphs, Matrix for dense graphs
Edge Lookup (u, v)	$O(1)$	$O(\deg(u))$	Matrix gives constant time
Iterate Neighbors	$O(V)$	$O(\deg(u))$	List is faster for iteration
Adding an Edge	$O(1)$	$O(1)$	Both are efficient
Removing an Edge	$O(1)$	$O(\deg(u))$	Matrix is faster
Best For	Dense Graph ($E \approx V^2$)	Sparse Graph ($E << V^2$)	Choose based on E and V

9. Graph Traversals

★ Why Graph Traversal ?

- Traversal means visiting every vertex of the graph.
- Used to search, explore, or process all vertices.
- Different traversal orders give different results.

Example Graph (Undirected)



Legend

|V| = 6 (Number of vertices)

|E| = 7 (Number of edges)

Edges : (1-2), (1-3), (2-4), (2-5), (3-5), (3-6), (4-5)

★ Types of Traversal

Traversal	Approach	Data Structure Used	Time Complexity	Best Suited For
1. BFS (Breadth First Search)	Visit all neighbors first, then move to next level.	Queue	O(V + E)	Shortest path in unweighted graph, level wise processing.
2. DFS (Depth First Search)	Go as deep as possible in one direction, then backtrack.	Stack / Recursion	O(V + E)	Cycle detection, Topological sort, Connected components.

★ 1. BFS (Breadth First Search)

- Explore all neighbors first.
- Use Queue.
- Visited level by level.

BFS Order

1, 2, 3, 4, 5, 6

BFS Step by Step (Start from 1)

Step	Queue	Visited
1	[1]	1
2	[2, 3]	1, 2, 3
3	[4, 5, 6]	1, 2, 3, 4, 5, 6
4	[]	1, 2, 3, 4, 5, 6

★ 2. DFS (Depth First Search)

- Go deep in one path.
- Use Stack or Recursion.
- Backtrack when no unvisited neighbor is found.

DFS Order

1, 2, 4, 5, 3, 6

DFS Step by Step (Start from 1)

Step	Stack	Visited
1	[1]	1
2	[2]	1, 2
3	[4]	1, 2, 4
4	[5]	1, 2, 4, 5
5	[3]	1, 2, 4, 5, 3
6	[6]	1, 2, 4, 5, 3, 6
7	[]	1, 2, 4, 5, 3, 6

★ BFS Implementation (Java)

```
void bfs(int start) {
  boolean[] visited = new boolean[V];
  Queue<Integer> q = new LinkedList<>();
  visited[start] = true;
  q.add(start);
  while(!q.isEmpty()) {
    int node = q.poll();
    System.out.print(node + " ");
    for (int neighbor : adj[node]) {
      if (!visited[neighbor]) {
        visited[neighbor] = true;
        q.add(neighbor);
      }
    }
  }
}
```

★ DFS Implementation (Java)

```
void dfs(int node, boolean[] visited) {
  visited[node] = true;
  System.out.print(node + " ");
  for (int neighbor : adj[node]) {
    if (!visited[neighbor]) {
      dfs(neighbor, visited);
    }
  }
}
```

How to call DFS ?

```
boolean[] visited = new boolean[V];
dfs(start, visited);
```

Key Points

- BFS uses Queue (FIFO).
- DFS uses Stack (LIFO) or Recursion.
- Both have same time complexity O(V + E).
- Choice depends on problem requirement.

★ Applications

✓ BFS → Shortest Path	Level Order Traversal	Web Crawling
✓ DFS → Topological Sort	Cycle Detection	Connected Components

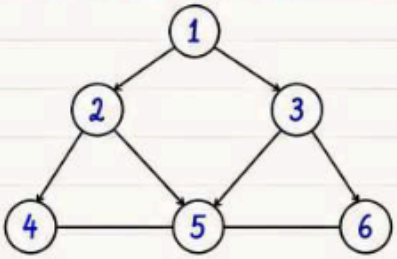
BFS goes wide, DFS goes deep !

10. Graph Algorithms (Basics)

★ Common Graph Algorithms

- BFS (Breadth First Search)
- DFS (Depth First Search)
- Shortest Path (Unweighted)
- Cycle Detection
- Connected Components

Example Graph (Undirected, Unweighted)



Graph Details

- $|V| = 6$ (Vertices)
- $|E| = 7$ (Edges)
- Edges : (1-2), (1-3), (2-4), (2-5), (3-5), (3-6), (4-5)
- Unweighted Graph

★ 1. BFS Traversal (Start from 1)

- Visit level by level using Queue.
- Order of visiting vertices : 1, 2, 3, 4, 5, 6

BFS Order

1, 2, 3, 4, 5, 6

BFS Step by Step

Step	Queue	Visited
1	[1]	1
2	[2, 3]	1, 2, 3
3	[4, 5, 6]	1, 2, 3, 4, 5, 6
4	[]	1, 2, 3, 4, 5, 6

★ 2. DFS Traversal (Start from 1)

- Go as deep as possible using Stack/Recursion.
- Order of visiting vertices : 1, 2, 4, 5, 3, 6

DFS Order

1, 2, 4, 5, 3, 6

DFS Step by Step (Using Stack)

Step	Stack	Visited
1	[1]	1
2	[2]	1, 2
3	[4, 5]	1, 2, 4, 5
4	[3, 6]	1, 2, 4, 5, 3, 6
5	[]	1, 2, 4, 5, 3, 6

★ 3. Shortest Path (Unweighted Graph)

- Use BFS to find shortest path.
- Find shortest path from 1 to 6.
- Shortest Path : 1 → 3 → 6

Node	Distance from 1
1	0
2	1
3	1
4	2
5	2
6	2

★ 4. Cycle Detection (Undirected Graph)

- If during DFS, we find a visited node which is not the parent, then cycle exists.

This graph contains cycle.

Cycle Found

Example Cycle : 2 → 4 → 5 → 2

★ 5. Connected Components

- A connected component is a set of vertices where each pair is connected.
- In this graph, all vertices are connected.

Connected Components

Total Components = 1

{1, 2, 3, 4, 5, 6}

★ Time Complexities

Algorithm	Time Complexity
BFS	O(V + E)
DFS	O(V + E)
Shortest Path (Unweighted)	O(V + E)
Cycle Detection	O(V + E)
Connected Components	O(V + E)

★ BFS in Java (Using Queue)

```
void bfs(int start) {
  boolean[] visited = new boolean[V];
  Queue<Integer> q = new LinkedList<>();
  visited[start] = true;
  q.add(start);
  while (!q.isEmpty()) {
    int node = q.poll();
    System.out.print(node + " ");
    for (int neighbor : adj[node]) {
      if (!visited[neighbor]) {
        visited[neighbor] = true;
        q.add(neighbor);
      }
    }
  }
}
```

★ DFS in Java (Using Recursion)

```
void dfs(int node, boolean[] visited) {
    visited[node] = true;
    System.out.print(node + " ");
    for (int neighbor : adj[node]) {
        if (!visited[neighbor]) {
            dfs(neighbor, visited);
        }
    }
}
```

★ BFS Shortest Path (From 1 to 6)

```
void shortestPath(int src, int dest) {
  int[] parent = new int[V];
  Arrays.fill(parent, -1);
  Queue<Integer> q = new LinkedList<>();
  boolean[] visited = new boolean[V];
  q.add(src);
  visited[src] = true;
  while (!q.isEmpty()) {
    int node = q.poll();
    if (node == dest) break;
    for (int neighbor : adj[node]) {
      if (!visited[neighbor]) {
        visited[neighbor] = true;
        parent[neighbor] = node;
        q.add(neighbor);
      }
    }
  }
  // reconstruct path from dest to src using parent[]
}
```

At a Glance

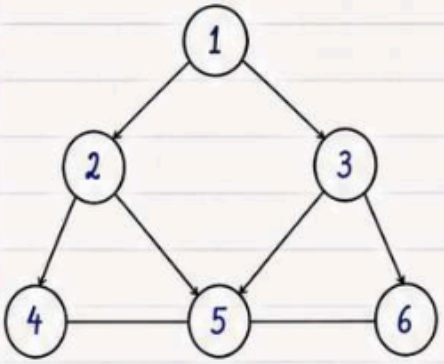
BFS	DFS	Shortest Path	Cycle Detection	Connected Components
Level by Level Uses Queue	Go Deep First Uses Stack / Recursion	Use BFS (Unweighted) Find minimum edges	Check for back edges in DFS	Find all reachable vertices from each node

11. Graph Algorithms (Part - 1)

★ BFS vs DFS

Feature	BFS	DFS
Strategy	Level by Level	Go Deep First
Data Structure	Queue (FIFO)	Stack / Recursion
Time Complexity	O(V + E)	O(V + E)
Space Complexity	O(V)	O(V)
Completeness	Yes (for finite graph)	May not be (infinite depth)
Optimality	Yes (unweighted)	No
Best Use	Shortest Path, Level Order	Topological Sort, Cycle Detect

Example Graph (Undirected)



Graph Details

- |V| = 6
- |E| = 7
- Edges : (1-2), (1-3), (2-4), (2-5), (3-5), (3-6), (4-5)
- Undirected Graph

★ 1. BFS Traversal (Start from 1)

- Visit level by level using Queue.
- Order of visiting nodes :

1, 2, 3, 4, 5, 6

BFS Step by Step

Step	Queue	Visited
1	[1]	1
2	[2, 3]	1, 2, 3
3	[4, 5, 6]	1, 2, 3, 4, 5, 6
4	[ ]	1, 2, 3, 4, 5, 6

BFS Order

1, 2, 3, 4, 5, 6

★ 2. DFS Traversal (Start from 1)

- Go as deep as possible using Stack / Recursion.
- Order of visiting nodes :

1, 2, 4, 5, 3, 6

DFS Step by Step (Using Stack)

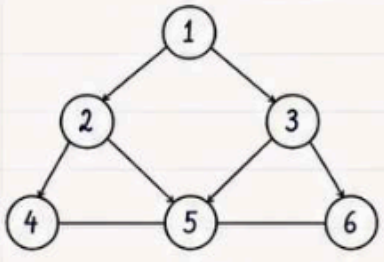
Step	Stack	Visited
1	[1]	1
2	[2]	1, 2
3	[4, 5]	1, 2, 4, 5
4	[3, 6]	1, 2, 4, 5, 3, 6
5	[ ]	1, 2, 4, 5, 3, 6

DFS Order

1, 2, 4, 5, 3, 6

★ 3. Shortest Path (Unweighted Graph)

- Use BFS to find shortest path.
- Find shortest path from 1 to 6.



Distance from 1

Node	Dist
1	0
2	1
3	1
4	2
5	2
6	2

Shortest Path : 1 → 3 → 6

Distance : 2

★ 4. Cycle Detection (Undirected Graph)

- If during DFS, we find a visited node which is not the parent, then cycle exists.

This graph contains cycle.

Example Cycle

2 → 4 → 5 → 2

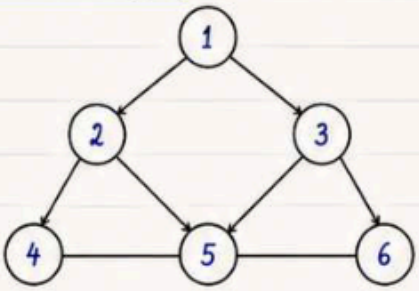
Idea

If we find a visited node which is not the parent of current node, then cycle exists.

11. Graph Algorithms (Part -2)

★ 5. Connected Components (Undirected Graph)

- A connected component is a set of vertices where each pair is connected.
- In this graph, all vertices are in one component.



Connected Components

Total Components = 1

{ 1, 2, 3, 4, 5, 6 }

Ways to Find Connected Components

Method	Approach
BFS Based	Run BFS from an unvisited node. Mark all visited. Repeat for all unvisited nodes.
DFS Based	Run DFS from an unvisited node. Mark all visited. Repeat for all unvisited nodes.

Connected Components in Java (Using BFS)

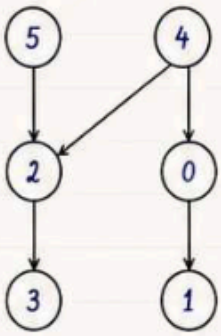
```
List<List<Integer>> components = new ArrayList<>();
boolean[] visited = new boolean[V];
for (int i = 1; i <= V; i++) {
    if (!visited[i]) {
        List<Integer> comp = new ArrayList<>();
        bfs(i, visited, comp);
        components.add(comp);
    }
}
// Each list in components is one connected component
```

★ 6. Topological Sort (For Directed Acyclic Graph - DAG)

Idea :

Order vertices such that for every directed edge $u \rightarrow v$, u comes before v in the order.

Example DAG



One Possible Topological Order

5 4 2 3 0 1

Topological Sort (Kahn's Algorithm - BFS Based)

```
Queue<Integer> q = new LinkedList<>();
int[] indegree = new int[V];
List<Integer> topo = new ArrayList<>();
// 1. Compute indegree
for (int u = 0; u < V; u++)
    for (int v : adj[u]) indegree[v]++;
// 2. Add all nodes with indegree 0
for (int i = 0; i < V; i++)
    if (indegree[i] == 0) q.add(i);
// 3. Process
while (!q.isEmpty()) {
   int node = q.poll();
   topo add(node);
   for (int nbr : adj[node]) {
      if (--indegree[nbr] == 0) q.add(nbr);
   }
}
// If topo size < V, then cycle exists
```

Summary of Graph Algorithms

Algorithm	Type	Main Idea	Data Structure	Time Complexity	Best Used For
BFS	Traversal	Visit level by level	Queue	O(V + E)	Shortest path (unweighted), Level order traversal
DFS	Traversal	Go as deep as possible	Stack / Recursion	O(V + E)	Cycle detection, Topological Sort, Connected Components
Shortest Path (Unweighted)	Path Finding	Use BFS	Queue	O(V + E)	Shortest path in unweighted graph
Cycle Detection (Undirected)	Detection	DFS with parent check	Stack / Recursion	O(V + E)	Check if cycle exists
Connected Components	Detection	BFS or DFS from each unvisited node	Queue / Stack	O(V + E)	Find all components in a graph
Topological Sort (DAG)	Ordering	Use Kahn's Algorithm	Queue	O(V + E)	Task scheduling, Dependency resolution

12. Graph Algorithms (Summary)

★ Key Takeaways

1. BFS explores level by level using Queue.
2. DFS explores as deep as possible using Stack / Recursion.
3. Shortest Path in unweighted graph can be found using BFS.
4. Cycle detection is possible using DFS.
5. Topological sort is only possible for DAG.

When to Use What ?

Problem Type	Use
Shortest Path (Unweighted)	BFS
Level Order Traversal	BFS
Cycle Detection (Undirected)	DFS
Cycle Detection (Directed)	DFS
Topological Sort	DFS (Kahn's Algorithm)
Connected Components	BFS or DFS

★ Complexity Summary

Algorithm	Time Complexity	Space Complexity	Data Structure Used
BFS	$O(V + E)$	$O(V)$	Queue
DFS (Iterative)	$O(V + E)$	$O(V)$	Stack
DFS (Recursive)	$O(V + E)$	$O(V)$	Recursion Stack
Shortest Path (Unweighted)	$O(V + E)$	$O(V)$	BFS (Queue)
Cycle Detection	$O(V + E)$	$O(V)$	DFS
Topological Sort (Kahn's)	$O(V + E)$	$O(V + E)$	Queue

★ Quick Revision

- BFS → Queue → Level by Level
- DFS → Stack / Recursion → Deep First
- Visited array → To avoid revisiting nodes.
- Graph can be represented using Adjacency List / Matrix.
- Unweighted graph shortest path → BFS.
- Cycle check
  - Undirected → DFS with parent check
  - Directed → DFS with recursion stack
- Topological sort → Only for DAG.
- Both BFS and DFS have same time complexity → $O(V + E)$.

★ Common Pitfalls

- Not marking node as visited → Leads to infinite loop.
- Using BFS for deep exploration problems.
- Forgetting parent in undirected cycle detection.
- Topological sort is not possible in cyclic graphs.
- Choosing wrong data structure may increase complexity.

Practice more problems to master Graph Algorithms !